

## Unit-5

### (I/O Management and Disk Scheduling)

#### Input Output (IO) Management

##### Issues in IO Management

Issues initially from the point of view of communication with a device we notice that communication is required at the following three levels:

- The need for a human to input information and receive output from a computer.
- The need for a device to input information and receive output from a computer.
- The need for computers to communicate (receive/send information) over networks.

The first kind of IO devices operate at rates good for humans to interact. These may be character-oriented devices like a keyboard or an event-generating device like a mouse.

The second kind of IO requirement arises from devices which have a very high character density such as tapes and disks. With these characteristics, it is not possible to regulate communication with devices on a character-by-character basis. The information transfer, therefore, is regulated in blocks of information.

The third kind of IO requirements emanate from the need to negotiate system IO with the communications infrastructure. The system should be able to manage communications traffic across the network. This form of IO facilitates access to internet resources to support e-mail, file-transfer amongst machines or Web applications.

#### IO Organization

Computers employ the following four basic modes of IO operation:

1. Programmed mode
2. Polling mode
3. Interrupt mode
4. Direct memory access mode.

##### Programmed Data Mode

In this mode of communication, execution of an IO instruction ensures that a program shall not advance till it is completed. To that extent one is assured that IO happens before anything else happens. As depicted in Figure 5.1, in this mode an IO instruction is issued to an IO device and the program executes in “busy-waiting” mode till the IO is completed. During the busy-wait period the processor is continually interrogating to check if the device has completed IO. Invariably the data transfer is accomplished through an identified register and a flag in a processor. For example, in Figure 5.1 depicts

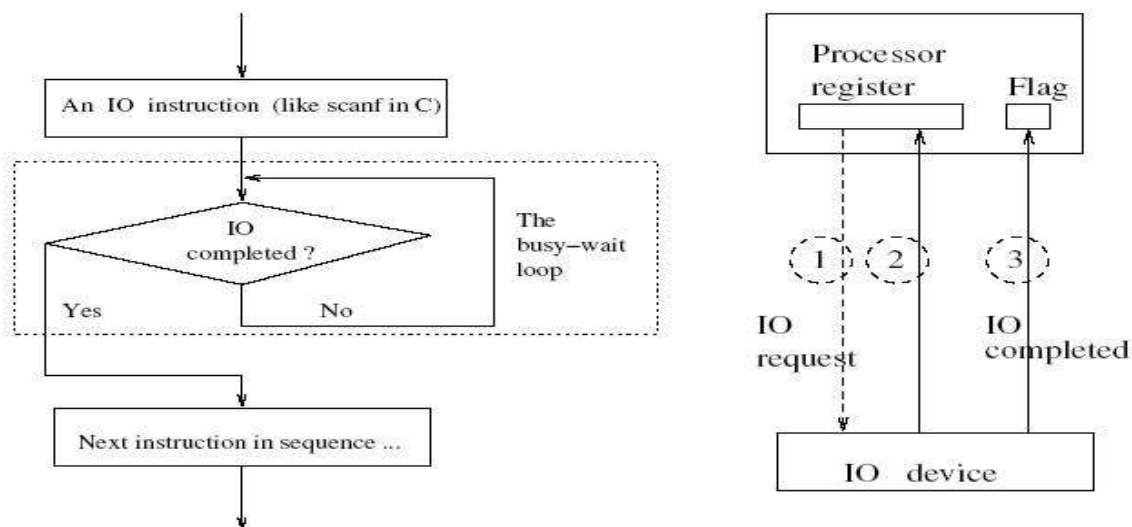


Figure 5.1: Programmed mode of IO.

When the IO is accomplished it signals the processor through the flag. During the busy-wait period the processor is busy checking the flag. However, the processor is idling from the point of view of doing anything useful. This situation is similar to a car engine which is running when the car is not in motion – essentially “idling”.

### Polling

In this mode of data transfer, shown in Figure 5.2, the system interrogates each device in turn to determine if it is ready to communicate. If it is ready, communication is initiated and subsequently the process continues again to interrogate in the same sequence. This is just like a round-robin strategy. Each IO device gets an opportunity to establish Communication in turn. No device has a particular advantage (like say a priority) over other devices.

Polling is quite commonly used by systems to interrogate ports on a network. Polling may also be scheduled to interrogate at some pre-assigned time intervals. It should be remarked here that most daemon software operate in polling mode. Essentially, they use a while true loop as shown in Figure 5.2.

In hardware, this may typically translate to the following protocol:

1. Assign a distinct address to each device connected to a bus.
2. The bus controller scans through the addresses in sequence to find which device wishes to establish a communication.
3. Allow the device that is ready to communicate to leave its data on the register.
4. The IO is accomplished. In case of an input the processor picks up the data. In case of an output the device picks up the data.
5. Move to interrogate the next device address in sequence to check if it is ready to communicate.

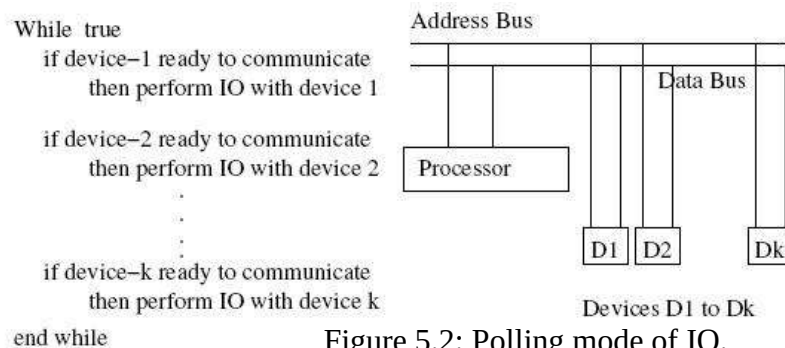


Figure 5.2: Polling mode of IO.

As we shall see next, polling may also be used within an interrupt service mode to identify the device which may have raised an interrupt.

### Interrupt Mode

To begin with, a program may initiate IO request and advance without suspending its operation. At the time when the device is actually ready to establish an IO, the device raises an interrupt to seek communication. Immediately the program execution is suspended temporarily and current state of the process is stored. The control is passed on to an interrupt service routine (which may be specific to the device) to perform the desired input. Subsequently, the suspended process context is restored to resume the program from the point of its suspension.

Interrupt processing may happen in the following contexts:

- **Internal Interrupt:** The source of interrupt may be a memory resident process or a function from within the processor. We regard such an interrupt as an internal interrupt.
- **External Interrupt:** If the source of interrupt is not internal, i.e. it is other than a process or processor related event then it is an external interrupt.
- **Software Interrupt:** Most OSs offer two modes of operation, the user mode and the system mode. Whenever a user program makes a system call, be it for IO or a special service, the operation must have a transition from user mode to system mode. An interrupt is raised to effect this transition from user to system mode of operation. Such an interrupt is called a software interrupt.

**Interrupt vector:** Many systems support an interrupt vector (IV). As depicted in

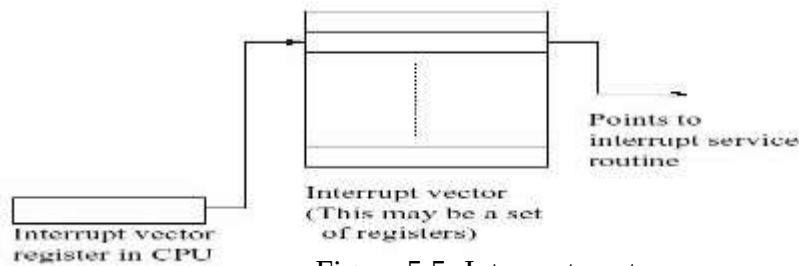


Figure 5.5: Interrupt vectors.

Figure 5.5, the basic idea revolves around an array of pointers to various interrupt service routines. Let us consider an example with four sources of interrupt. These may be a trap, a system call, an IO, or an interrupt initiated by a program. Now we may associate an index value 0 with trap, 1 with system call, 2 with IO device and 3 with the program interrupt. Note that the source of interrupt provides us the index in the vector. The interrupt service can now be provided as follows:

- Identify the source of interrupt and generate index  $i$ .
- Identify the interrupt service routine address by looking up  $IVR(i)$ , where IVR stands for the interrupt vector register. Let this address be  $ISR_i$ .
- Transfer control to the interrupt service routine by setting the program counter to  $ISR_i$ .

Note that the interrupt vector may also be utilized in the context of a priority-based interrupt in which the bit set in a bit vector determines the interrupt service routine to be selected. It is very easy to implement this in hardware.

### DMA Mode of Data Transfer

This is a mode of data transfer in which IO is performed in large data blocks. For instance, the disks communicate in data blocks of sizes like 512 bytes or 1024 bytes. The direct memory access, or DMA ensures access to main memory without processor intervention or support. Such independence from processor makes this mode of transfer extremely efficient.

When a process initiates a direct memory access (DMA) transfer, its execution is briefly suspended (using an interrupt) to set up the DMA control. The DMA control requires the information on starting address in main memory and size of data for transfer. This information is stored in DMA controller. Following the DMA set up, the program resumes from the point of suspension. The device communicates with main memory stealing memory access cycles in competition with other devices and processor. Figure 5.6 shows the hardware support.

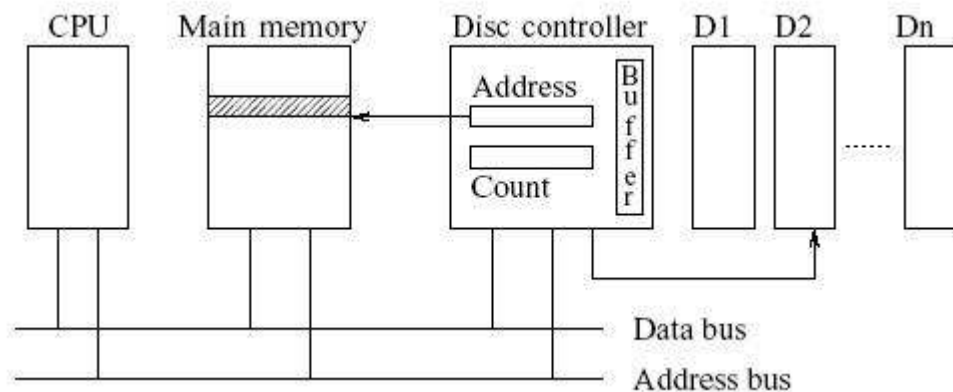


Figure 5.6: DMA : Hardware support.

**Spooling:** Suppose we have a printer connected to a machine. Many users may seek to use the printer. To avoid print clashes, it is important to be able to queue up all the print requests. This is achieved by spooling. The OS maintains all print requests and schedules each users' print requests. In other words, all output commands to print are intercepted by the OS kernel. An area is used to spool the output so that a users' job does not have to wait for the printer to be available.

# I/O Subsystems

The kernel provides many services related to I/O. Several services such as scheduling, caching, spooling, device reservation, and error handling – are provided by the kernel, I/O subsystem built on the hardware and device-driver infrastructure. The I/O subsystem is also responsible for protecting itself from errant processes and malicious users.

## 1. I/O Scheduling –

To schedule a set of I/O requests means to determine a good order in which to execute them. The order in which the application issues the system call is the best choice. Scheduling can improve the overall performance of the system, can share device access permission fairly to all the processes, reduce the average waiting time, response time, turnaround time for I/O to complete.

OS developers implement schedules by maintaining a wait queue of the request for each device. When an application issue a blocking I/O system call, The request is placed in the queue for that device. The I/O scheduler rearranges the order to improve the efficiency of the system.

## 2. Buffering –

A *buffer* is a memory area that stores data being transferred between two devices or between a device and an application. Buffering is done for three reasons.

1. The first is to cope with a speed mismatch between producer and consumer of a data stream.
2. The second use of buffering is to provide adaptation for data that have different data-transfer sizes.
3. The third use of buffering is to support copy semantics for the application I/O, “copy semantic ” means, suppose that an application wants to write data on a disk that is stored in its buffer. it calls the **write()** system’s call, providing a pointer to the buffer and the integer specifying the number of bytes to write.

## 3. Caching –

A *cache* is a region of fast memory that holds a copy of data. Access to the cached copy is much easier than the original file. For instance, the instruction of the currently running process is stored on the disk, cached in physical memory, and copied again in the CPU’s secondary and primary cache.

The main difference between a buffer and a cache is that a buffer may hold only the existing copy of a data item, while a cache, by definition, holds a copy on faster storage of an item that resides elsewhere.

## 4. Spooling and Device Reservation –

A *spool* is a buffer that holds the output of a device, such as a printer that cannot accept interleaved data streams. Although a printer can serve only one job at a time, several applications may wish to print their output concurrently, without having their output mixes together.

The OS solves this problem by preventing all output from continuing to the printer. The output of all applications is spooled in a separate disk file. When an application finishes printing then the spooling system queues the corresponding spool file for output to the printer.

## 5. Error Handling –

An Os that uses protected memory can guard against many kinds of hardware and application errors so that a complete system failure is not the usual result of each minor mechanical glitch, Devices, and I/O transfers can fail in many ways, either for transient reasons, as when a network becomes overloaded or for permanent reasons, as when a disk controller becomes defective.

## 6. I/O Protection –

Errors and the issue of protection are closely related. A user process may attempt to issue illegal I/O instructions to disrupt the normal function of a system. We can use the various mechanisms to ensure that such disruption cannot take place in the system.

To prevent illegal I/O access, we define all I/O instructions to be privileged instructions. The user cannot issue I/O instruction directly.

# I/O Buffering

A buffer is a memory area that stores data being transferred between two devices or between a device and an application.

## Uses of I/O Buffering :

- Buffering is done to deal effectively with a speed mismatch between the producer and consumer of the data stream.
- A buffer is produced in main memory to heap up the bytes received from modem.
- After receiving the data in the buffer, the data get transferred to disk from buffer in a single operation.
- This process of data transfer is not instantaneous; therefore the modem needs another buffer in order to store additional incoming data.
- When the first buffer got filled, then it is requested to transfer the data to disk.
- The modem then starts filling the additional incoming data in the second buffer while the data in the first buffer getting transferred to disk.
- When both the buffers completed their tasks, then the modem switches back to the first buffer while the data from the second buffer get transferred to the disk.
- The use of two buffers disintegrates the producer and the consumer of the data, thus minimizes the time requirements between them.
- Buffering also provides variations for devices that have different data transfer sizes.

## Types of various I/O buffering techniques :

### 1. Single buffer :

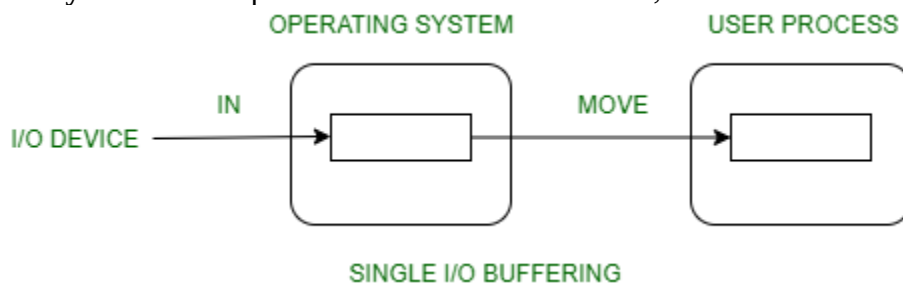
A buffer is provided by the operating system to the system portion of the main memory.

#### Block oriented device –

- System buffer takes the input.
- After taking the input, the block gets transferred to the user space by the process and then the process requests for another block.
- Two blocks works simultaneously, when one block of data is processed by the user process, the next block is being read in.
- OS can swap the processes.
- OS can record the data of system buffer to user processes.

#### Stream oriented device –

- **Line-** at a time operation is used for scroll made terminals. User inputs one line at a time, with a carriage return signaling at the end of a line.
- **Byte-** at a time operation is used on forms mode, terminals when each keystroke is significant.



### 2. Double buffer :

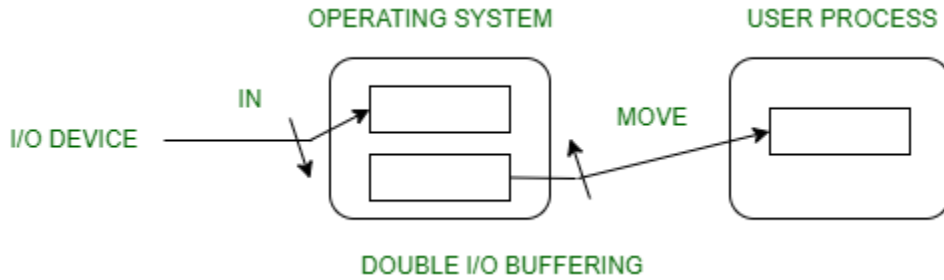
#### Block oriented –

- There are two buffers in the system.
- One buffer is used by the driver or controller to store data while waiting for it to be taken by higher level of the hierarchy.
- Other buffer is used to store data from the lower level module.
- Double buffering is also known as buffer swapping.
- A major disadvantage of double buffering is that the complexity of the process get increased.

- If the process performs rapid bursts of I/O, then using double buffering may be deficient.

#### Stream oriented –

- Line- at a time I/O, the user process need not be suspended for input or output, unless process runs ahead of the double buffer.
- Byte- at a time operations, double buffer offers no advantage over a single buffer of twice the length.



#### 3. Circular buffer :

- When more than two buffers are used, the collection of buffers is itself referred to as a circular buffer.
- In this, the data do not directly passed from the producer to the consumer because the data would change due to overwriting of buffers before they had been consumed.
- The producer can only fill up to buffer i-1 while data in buffer i is waiting to be consumed.

## Disk Storage

Basically we want the programs and data to reside in main memory permanently.

This arrangement is usually not possible for the following two reasons:

1. Main memory is usually too small to store all needed programs and data permanently.
2. Main memory is a volatile storage device that loses its contents when power is turned off or otherwise lost.

There are two types of storage devices:-

#### • Volatile Storage Device –

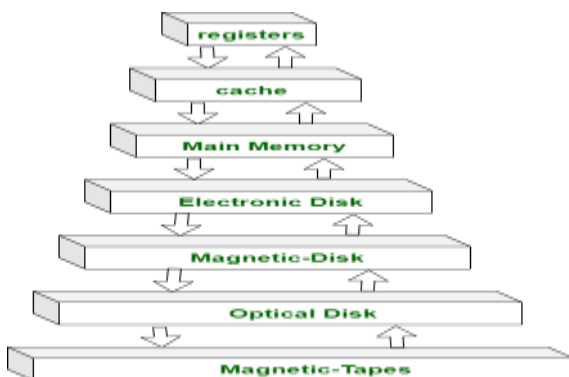
It loses its contents when the power of the device is removed.

#### • Non-Volatile Storage device –

It does not lose its contents when the power is removed. It holds all the data when the power is removed.

Secondary Storage is used as an extension of main memory. Secondary storage devices can hold the data permanently.

Storage devices consist of Registers, Cache, Main-Memory, Electronic-Disk, Magnetic-Disk, Optical-Disk, Magnetic-Tapes. Each storage system provides the basic system of storing a datum and of holding the datum until it is retrieved at a later time. All the storage devices differ in speed, cost, size and volatility. The most common Secondary-storage device is a Magnetic-disk, which provides storage for both programs and data.



Storage Device Hierarchy.

fig.: Hierarchy of storage

In this hierarchy all the storage devices are arranged according to speed and cost. The higher levels are expensive, but they are fast. As we move down the hierarchy, the cost per bit generally decreases, where as the access time generally increases.

The storage systems above the Electronic disk are Volatile, where as those below are Non-Volatile. An Electronic disk can be either designed to be either Volatile or Non-Volatile. During normal operation, the electronic disk stores data in a large DRAM array, which is Volatile. But many electronic disk devices contain a hidden magnetic hard disk and a battery for backup power. If external power is interrupted, the electronic disk controller copies the data from RAM to the magnetic disk. When external power is restored, the controller copies the data back into the RAM.

The design of a complete memory system must balance all the factors. It must use only as much expensive memory as necessary while providing as much inexpensive, Non-Volatile memory as possible. Caches can be installed to improve performance where a large access-time or transfer-rate disparity exists between two components.

## Disk Scheduling-

Disk scheduling is a technique used by the operating system to schedule multiple requests for accessing the disk

## Disk Scheduling Algorithms-

- The algorithms used for disk scheduling are called as **disk scheduling algorithms**.
- The purpose of disk scheduling algorithms is to reduce the total seek time.

Various disk scheduling algorithms are-



1. FCFS Algorithm
2. SSTF Algorithm
3. SCAN Algorithm
4. C-SCAN Algorithm
5. LOOK Algorithm
6. C-LOOK Algorithm

## FCFS Disk Scheduling Algorithm-

- As the name suggests, this algorithm entertains requests in the order they arrive in the disk queue.
- It is the simplest disk scheduling algorithm.

## Advantages-

- It is simple, easy to understand and implement.
- It does not cause starvation to any request.

## Disadvantages-

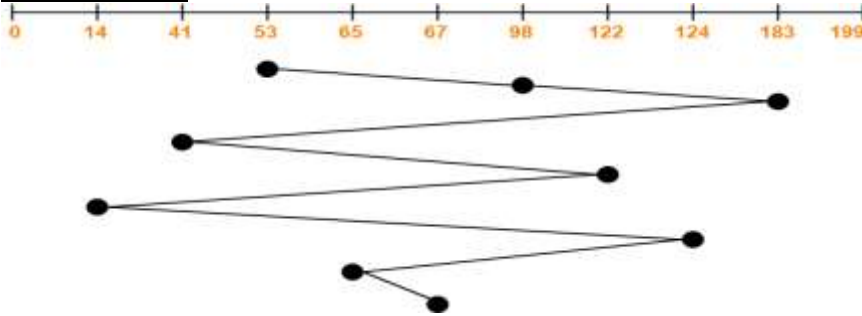
- It results in increased total seek time.
- It is inefficient.

## PRACTICE PROBLEM BASED ON FCFS DISK SCHEDULING ALGORITHM-

## Problem-

Consider a disk queue with requests for I/O to blocks on cylinders 98, 183, 41, 122, 14, 124, 65, 67. The FCFS scheduling algorithm is used. The head is initially at cylinder number 53. The cylinders are numbered from 0 to 199. The total head movement (in number of cylinders) incurred while servicing these requests is

## Solution-



Total head movements incurred while servicing these requests

$$\begin{aligned} &= (98 - 53) + (183 - 98) + (183 - 41) + (122 - 41) + (122 - 14) + (124 - 14) + (124 - 65) + (67 - 65) \\ &= 45 + 85 + 142 + 81 + 108 + 110 + 59 + 2 \\ &= 632 \end{aligned}$$

## SSTF Disk Scheduling Algorithm-

- SSTF stands for **Shortest Seek Time First**.
- This algorithm services that request next which requires least number of head movements from its current position regardless of the direction.
- It breaks the tie in the direction of head movement.

## Advantages-

- It reduces the total seek time as compared to FCFS.
- It provides increased throughput.
- It provides less average response time and waiting time.

## Disadvantages-

- There is an overhead of finding out the closest request.
- The requests which are far from the head might starve for the CPU.
- It provides high variance in response time and waiting time.
- Switching the direction of head frequently slows down the algorithm.

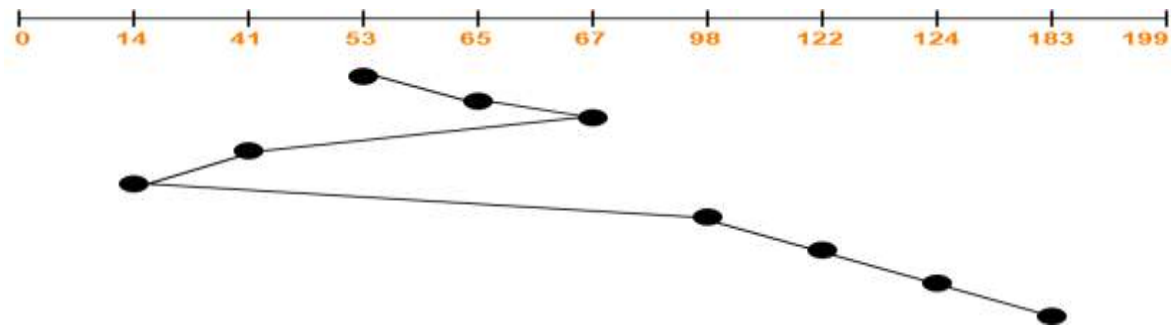
## PRACTICE PROBLEMS BASED ON SSTF DISK SCHEDULING ALGORITHM-

### Problem-01:

Consider a disk queue with requests for I/O to blocks on cylinders 98, 183, 41, 122, 14, 124, 65, 67. The SSTF scheduling algorithm is used. The head is initially at cylinder number 53 moving towards larger cylinder numbers on its servicing pass. The cylinders are numbered from 0 to 199. The total head movement (in number of cylinders) incurred while servicing these requests is \_\_\_\_\_.

## Solution-





Total head movements incurred while servicing these requests

$$\begin{aligned}
 &= (65 - 53) + (67 - 65) + (67 - 41) + (41 - 14) + (98 - 14) + (122 - 98) + (124 - 122) + (183 - 124) \\
 &= 12 + 2 + 26 + 27 + 84 + 24 + 2 + 59 \\
 &= 236
 \end{aligned}$$

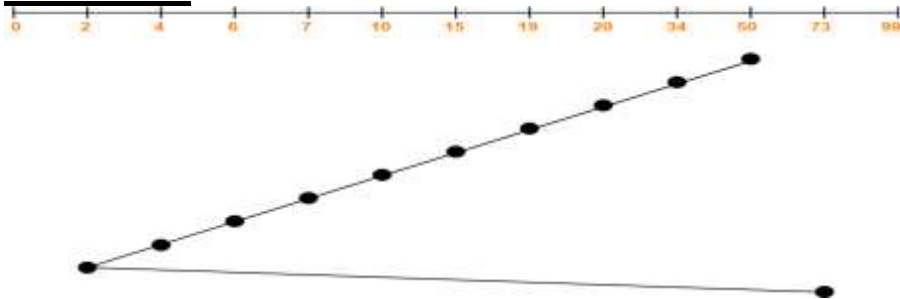
## **Problem-02:**

Consider a disk system with 100 cylinders. The requests to access the cylinders occur in following sequence-  
4, 34, 10, 7, 19, 73, 2, 15, 6, 20

Assuming that the head is currently at cylinder 50, what is the time taken to satisfy all requests if it takes 1 ms to move from one cylinder to adjacent one and shortest seek time first policy is used?

1. 95 ms
2. 119 ms
3. 233 ms
4. 276 ms

## **Solution-**



Total head movements incurred while servicing these requests

$$\begin{aligned}
 &= (50 - 34) + (34 - 20) + (20 - 19) + (19 - 15) + (15 - 10) + (10 - 7) + (7 - 6) + (6 - 4) + (4 - 2) + (73 - 2) \\
 &= 16 + 14 + 1 + 4 + 5 + 3 + 1 + 2 + 2 + 71 \\
 &= 119
 \end{aligned}$$

Time taken for one head movement = 1 msec. So,

Time taken for 119 head movements

$$= 119 \times 1 \text{ msec}$$

$$= 119 \text{ msec}$$

Thus, Option (B) is correct.

## **SCAN Disk Scheduling Algorithm-**

As the name suggests, this algorithm scans all the cylinders of the disk back and forth.

- Head starts from one end of the disk and move towards the other end servicing all the requests in between.
- After reaching the other end, head reverses its direction and move towards the starting end servicing all the requests in between.
- The same process repeats.

## **NOTE-**

- SCAN Algorithm is also called as **Elevator Algorithm**.
- This is because its working resembles the working of an elevator.

### Advantages-

- It is simple, easy to understand and implement.
- It does not lead to starvation.
- It provides low variance in response time and waiting time.

### Disadvantages-

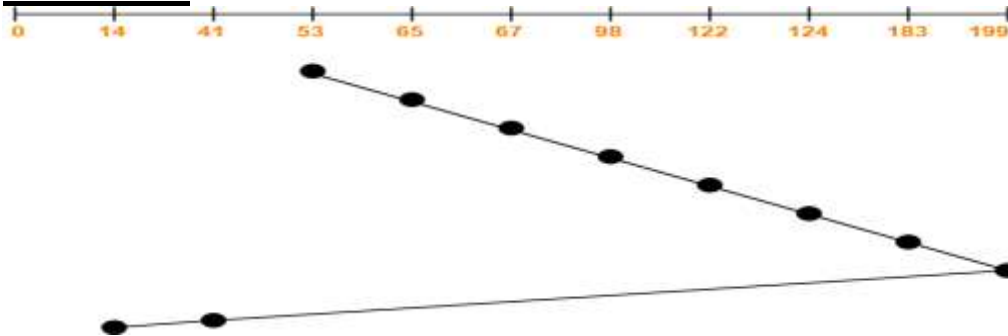
- It causes long waiting time for the cylinders just visited by the head.
- It causes the head to move till the end of the disk even if there are no requests to be serviced.

## PRACTICE PROBLEM BASED ON SCAN DISK SCHEDULING ALGORITHM-

### Problem-

Consider a disk queue with requests for I/O to blocks on cylinders 98, 183, 41, 122, 14, 124, 65, 67. The SCAN scheduling algorithm is used. The head is initially at cylinder number 53 moving towards larger cylinder numbers on its servicing pass. The cylinders are numbered from 0 to 199. The total head movement (in number of cylinders) incurred while servicing these requests is \_\_\_\_\_.

### Solution-



Total head movements incurred while servicing these requests  
 $= (65 - 53) + (67 - 65) + (98 - 67) + (122 - 98) + (124 - 122) + (183 - 124) + (199 - 183) + (199 - 41) + (41 - 14)$   
 $= 12 + 2 + 31 + 24 + 2 + 59 + 16 + 158 + 27 = 331$

Alternatively,

Total head movements incurred while servicing these requests  
 $= (199 - 53) + (199 - 14)$   
 $= 146 + 185$   
 $= 331$

## C-SCAN Disk Scheduling Algorithm-

- Circular-SCAN Algorithm is an improved version of the SCAN Algorithm.
- Head starts from one end of the disk and move towards the other end servicing all the requests in between.
- After reaching the other end, head reverses its direction.
- It then returns to the starting end without servicing any request in between.
- The same process repeats.

### Advantages-

- The waiting time for the cylinders just visited by the head is reduced as compared to the SCAN Algorithm.
- It provides uniform waiting time.
- It provides better response time.

### Disadvantages-

- It causes more seek movements as compared to SCAN Algorithm.

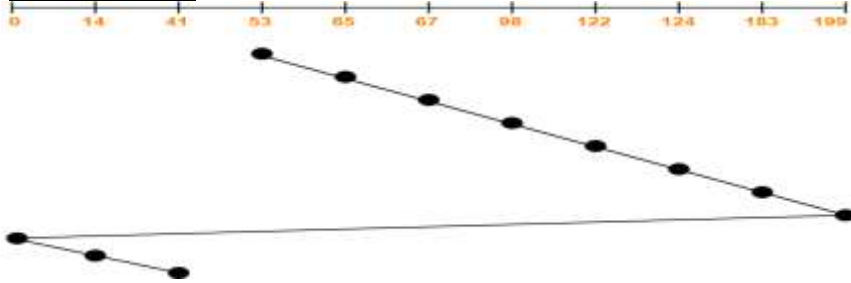
- It causes the head to move till the end of the disk even if there are no requests to be serviced.

## **PRACTICE PROBLEM BASED ON C-SCAN DISK SCHEDULING ALGORITHM-**

### **Problem-**

Consider a disk queue with requests for I/O to blocks on cylinders 98, 183, 41, 122, 14, 124, 65, 67. The C-SCAN scheduling algorithm is used. The head is initially at cylinder number 53 moving towards larger cylinder numbers on its servicing pass. The cylinders are numbered from 0 to 199. The total head movement (in number of cylinders) incurred while servicing these requests is \_\_\_\_\_.

### **Solution-**



Total head movements incurred while servicing these requests

$$\begin{aligned}
 &= (65 - 53) + (67 - 65) + (98 - 67) + (122 - 98) + (124 - 122) + (183 - 124) + (199 - 183) + (199 - 0) + (14 - 0) + (41 - 14) \\
 &= 12 + 2 + 31 + 24 + 2 + 59 + 16 + 199 + 14 + 27 \\
 &= 386
 \end{aligned}$$

**Alternatively,**

Total head movements incurred while servicing these requests

$$\begin{aligned}
 &= (199 - 53) + (199 - 0) + (41 - 0) \\
 &= 146 + 199 + 41 \\
 &= 386
 \end{aligned}$$

## **LOOK Disk Scheduling Algorithm-**

LOOK Algorithm is an improved version of the SCAN Algorithm.

- Head starts from the first request at one end of the disk and moves towards the last request at the other end servicing all the requests in between.
- After reaching the last request at the other end, head reverses its direction.
- It then returns to the first request at the starting end servicing all the requests in between.
- The same process repeats.

### **NOTE-**

The main difference between SCAN Algorithm and LOOK Algorithm is-

- SCAN Algorithm scans all the cylinders of the disk starting from one end to the other end even if there are no requests at the ends.
- LOOK Algorithm scans all the cylinders of the disk starting from the first request at one end to the last request at the other end.

### **Advantages-**

- It does not causes the head to move till the ends of the disk when there are no requests to be serviced.
- It provides better performance as compared to SCAN Algorithm.
- It does not lead to starvation.
- It provides low variance in response time and waiting time.

### **Disadvantages-**

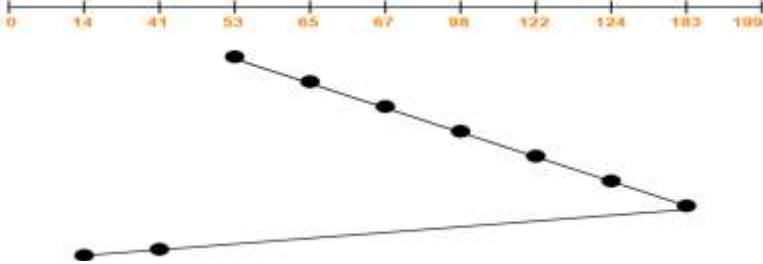
- There is an overhead of finding the end requests.
- It causes long waiting time for the cylinders just visited by the head.

## **PRACTICE PROBLEM BASED ON LOOK DISK SCHEDULING ALGORITHM-**

### **Problem-**

Consider a disk queue with requests for I/O to blocks on cylinders 98, 183, 41, 122, 14, 124, 65, 67. The LOOK scheduling algorithm is used. The head is initially at cylinder number 53 moving towards larger cylinder numbers on its servicing pass. The cylinders are numbered from 0 to 199. The total head movement (in number of cylinders) incurred while servicing these requests is \_\_\_\_\_.

### **Solution-**



Total head movements incurred while servicing these requests  
=  $(65 - 53) + (67 - 65) + (98 - 67) + (122 - 98) + (124 - 122) + (183 - 124) + (183 - 41) + (41 - 14)$   
=  $12 + 2 + 31 + 24 + 2 + 59 + 142 + 27$   
= 299

## **C-LOOK Disk Scheduling Algorithm-**

Circular-LOOK Algorithm is an improved version of the LOOK Algorithm.

- Head starts from the first request at one end of the disk and moves towards the last request at the other end servicing all the requests in between.
- After reaching the last request at the other end, head reverses its direction.
- It then returns to the first request at the starting end without servicing any request in between.
- The same process repeats.

### **Advantages-**

- It does not cause the head to move till the ends of the disk when there are no requests to be serviced.
- It reduces the waiting time for the cylinders just visited by the head.
- It provides better performance as compared to LOOK Algorithm.
- It does not lead to starvation.
- It provides low variance in response time and waiting time.

### **Disadvantages-**

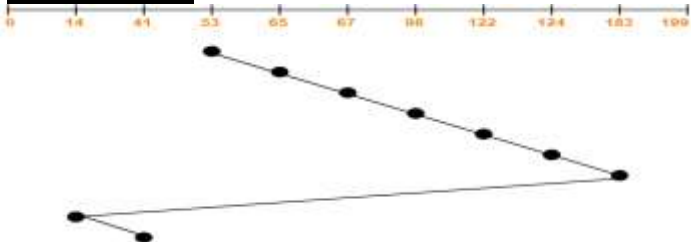
- There is an overhead of finding the end requests.

## **PRACTICE PROBLEMS BASED ON C-LOOK DISK SCHEDULING ALGORITHM-**

### **Problem-01:**

Consider a disk queue with requests for I/O to blocks on cylinders 98, 183, 41, 122, 14, 124, 65, 67. The C-LOOK scheduling algorithm is used. The head is initially at cylinder number 53 moving towards larger cylinder numbers on its servicing pass. The cylinders are numbered from 0 to 199. The total head movement (in number of cylinders) incurred while servicing these requests is \_\_\_\_\_.

### **Solution-**



Total head movements incurred while servicing these requests

$$\begin{aligned}
 &= (65 - 53) + (67 - 65) + (98 - 67) + (122 - 98) + (124 - 122) + (183 - 124) + (183 - 14) + (41 - 14) \\
 &= 12 + 2 + 31 + 24 + 2 + 59 + 169 + 27 \\
 &= 326
 \end{aligned}$$

**Alternatively,**

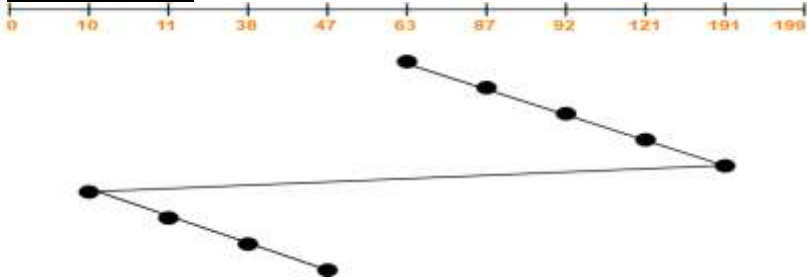
Total head movements incurred while servicing these requests

$$\begin{aligned}
 &= (183 - 53) + (183 - 14) + (41 - 14) \\
 &= 130 + 169 + 27 \\
 &= 326
 \end{aligned}$$

## **Problem-02:**

Consider a disk queue with requests for I/O to blocks on cylinders 47, 38, 121, 191, 87, 11, 92, 10. The C-LOOK scheduling algorithm is used. The head is initially at cylinder number 63 moving towards larger cylinder numbers on its servicing pass. The cylinders are numbered from 0 to 199. The total head movement (in number of cylinders) incurred while servicing these requests is \_\_\_\_\_.

## **Solution-**



Total head movements incurred while servicing these requests

$$\begin{aligned}
 &= (87 - 63) + (92 - 87) + (121 - 92) + (191 - 121) + (191 - 10) + (11 - 10) + (38 - 11) + (47 - 38) \\
 &= 24 + 5 + 29 + 70 + 181 + 1 + 27 + 9 \\
 &= 346
 \end{aligned}$$

**Alternatively,**

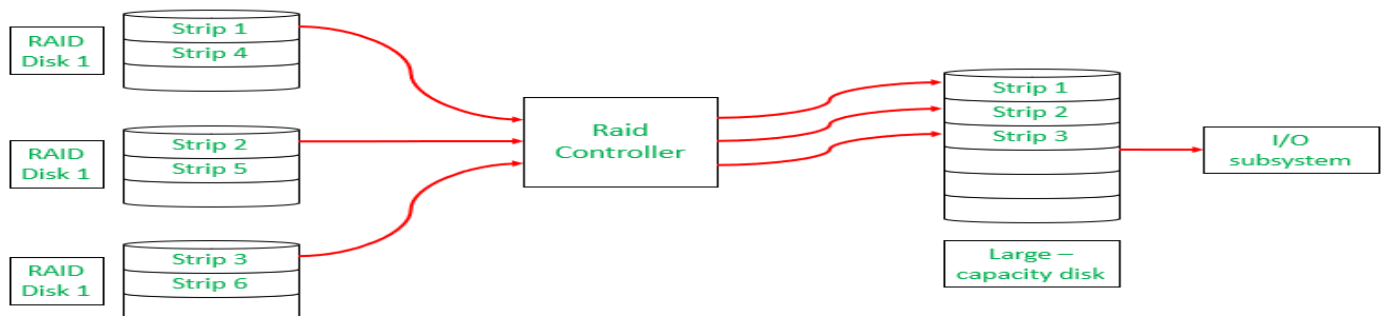
Total head movements incurred while servicing these requests

$$\begin{aligned}
 &= (191 - 63) + (191 - 10) + (47 - 10) \\
 &= 128 + 181 + 37 \\
 &= 346
 \end{aligned}$$

## **Redundant Array of Independent Disks (RAID) :**

**Redundant Array of Independent Disks (RAID)** is a set of several physical disk drives that Operating System see as a single logical unit. It played a significant role in narrowing the gap between increasingly fast processors and slow disk drives.

The basic principle behind RAID is that several smaller-capacity disk drives are better in performance than some large-capacity disk drives because through distributing the data among several smaller disks, the system can access data from them faster, resulting in improved I/O performance and improved data recovery in case of disk failure.



A typical disk array configuration consists of small disk drives connected to a controller housing the software and coordinating the transfer of data in the disks to a large capacity disk connected to I/O

subsystem.

Note that this whole configuration is viewed as a single large-capacity disk by the OS.

- Data is divided into segments called **strips**, which are distributed across the disks in the array.
- A set of consecutive strips across the disks is called a **stripe**.
- The whole process is called **striping**.

Besides introducing the concept of redundancy which helps in data recovery due to hardware failure, it also increases the cost of the hardware.

The whole system of RAID is divided in seven levels from level 0 to level 6. Here, the level does not indicate hierarchy, but indicate different types of configurations and error correction capabilities.

#### Level 0 :

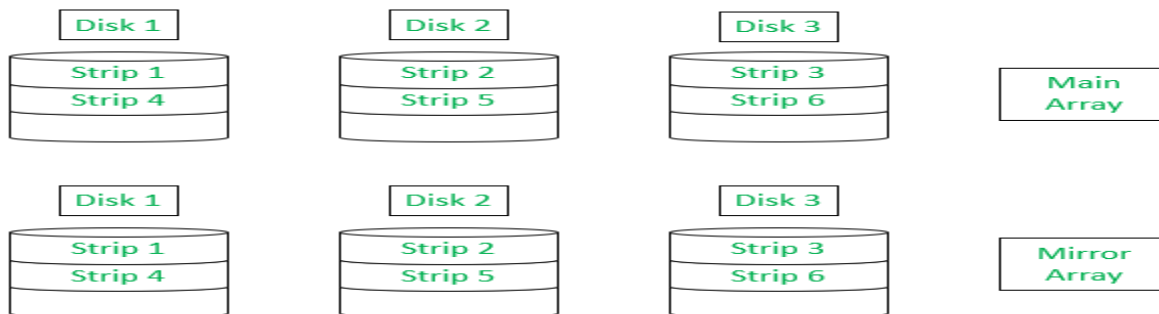
RAID level 0 is the only level that cannot recover from hardware failure, as it doesn't provide error correction or redundancy. Therefore, it can't be called the true form of RAID. However, it sure offers the same significant benefits as others – to the OS this group of devices appears to be a single logical unit.



As illustrated above, when the OS issues a command, that can be transferred in parallel to the strips, improving performance greatly.

#### Level 1 :

RAID level 1 not only uses the process of striping, but also uses mirrored configuration by providing redundancy, i.e., it creates a duplicate set of all the data in a mirrored array of disks, which as a backup in case of hardware failure. If one drive fails, data can be retrieved immediately from the mirrored array of disks. With this, it becomes a reliable system.



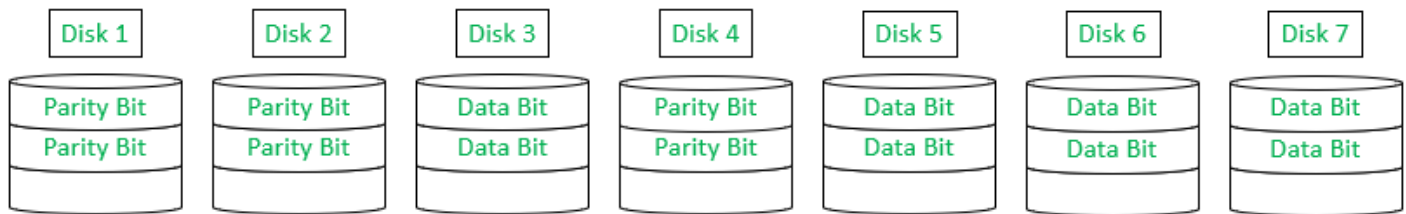
As illustrated above data has been copied in yet another array of disk as a backup.

- The **disadvantage** includes the writing of the data twice, once in main disks, and then in backup disks. However, process time can be saved by doing the copying the data in parallel to the main writing of the data.
- Another **disadvantage** is that it requires double amount of space, and so is expensive. But, the **advantage** of having a backup and no worry for data loss nullifies this disadvantage.

#### Level 2 :

RAID level 2 makes the use of very small strips (often of the size of 1 byte) and a **hamming code** to provide redundancy (for the task of error detection, correction, etc.).

**Hamming Code :** It is an algorithm used for error detection and correction when the data is being transferred. It adds extra, redundant bits to the data. It is able to correct single-bit errors and correct double-bit errors.



This configuration has a **disadvantage** that it is an expensive and a complex configuration to implement because of the number of additional arrays, which depend on the size of strips, and also all the drives must be highly synchronized.

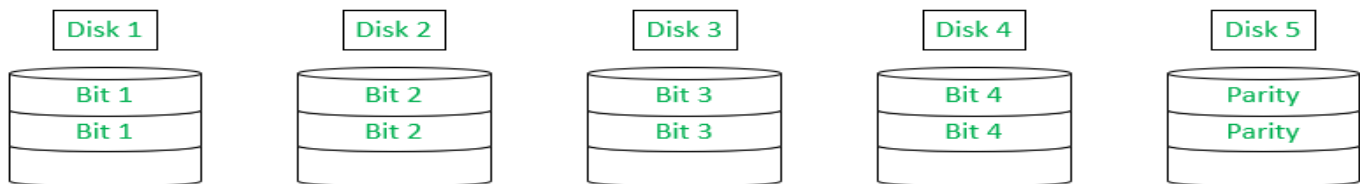
The **advantage** includes that if a drive should malfunction, then only one disk would be affected and the data could be quickly recovered.

### Level 3 :

RAID level 3 is a configuration that only needs one disk for redundancy. Only one parity bit is computed for each strip and is stored in designated redundant disk.

If a drive malfunctions, the RAID controller considers all the bits coming from that disk to be 0 and notes the location of that malfunctioning disk. So, if the data being read has a parity error, then the controller knows that the bit should be 1 and corrects it.

If data is being written to the array that has a malfunctioning device, then the controller keeps the parity consistent so as to regenerate data when the array is replaced. The system returns to normal when the failed disk is replaced and it's contents are regenerated on the new disk(or array).



### Level 4 :

RAID level 4 uses the same concept used in level 0 & level 1, but also computes a parity for each strip and stores this parity in the corresponding strip of the parity disk.

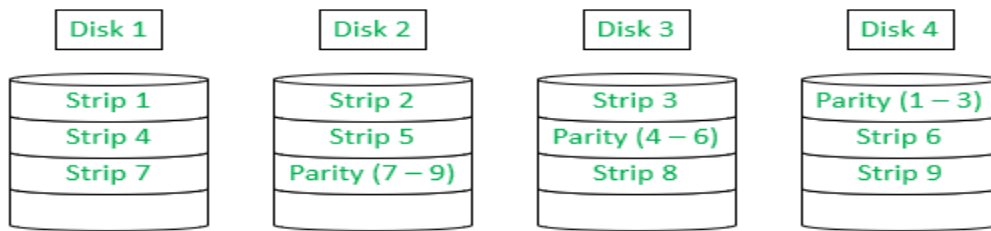
The **advantage** of this configuration is that if a disk fails, the data can be still recovered from the parity disk.

Parity is computed every time a write command is executed. But, when the data is to be rewritten inside the disks, the RAID controller must be able to update the data and parity disks. So, the parity disks need to be accessed whenever a write or rewrite operations are to be executed. This creates a situation known as the **bottleneck** which is the main **disadvantage** of this configuration.



### Level 5 :

RAID level 5 is a modification level 4. In level 4, only one disk is designated for parity storing parities. But in level 5, it distributes the parity disks across the disks in the array.



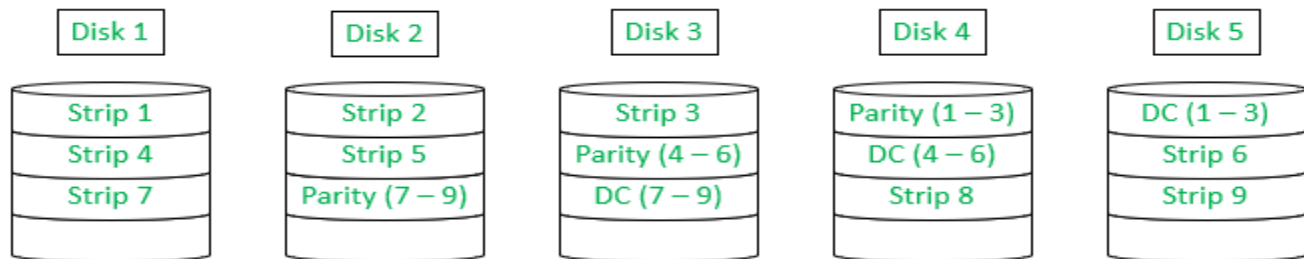
The **advantage** of this configuration is that it avoids the condition of bottleneck which was created in level 4.

The **disadvantage** of this configuration is that during regeneration of data when a disk fails is complicated.

#### Level 6 :

RAID level 6 provides an extra degree of error detection and correction. It requires 2 different parity calculations.

One calculation is the same as the one used level 4 and 5, other calculation is an independent data-check algorithm. Both the parities are stored on separate disks across the array, which corresponds to the data strips in the array.



The **advantage** of this configuration is that if even 2 disks fails or malfunction, then also the data can be recovered.

The **disadvantage** of this configuration includes:

- The redundancy increases the time required to write the data because now the data is to be also written on the second parity disk.
- In this configuration another disk is designated as the parity disk, which decreases the number of data disks in the array.

## File Concepts

### File

A file is a named collection of related information that is recorded on secondary storage such as magnetic disks, magnetic tapes and optical disks. In general, a file is a sequence of bits, bytes, lines or records whose meaning is defined by the files creator and user.

### File Structure

A File Structure should be according to a required format that the operating system can understand.

- A file has a certain defined structure according to its type.
- A text file is a sequence of characters organized into lines.
- A source file is a sequence of procedures and functions.
- An object file is a sequence of bytes organized into blocks that are understandable by the machine.
- When operating system defines different file structures, it also contains the code to support these file structure. Unix, MS-DOS support minimum number of file structure.



## File Type

File type refers to the ability of the operating system to distinguish different types of file such as text files source files and binary files etc. Many operating systems support many types of files. Operating system like MS-DOS and UNIX have the following types of files –

### Ordinary files

- These are the files that contain user information.
- These may have text, databases or executable program.
- The user can apply various operations on such files like add, modify, delete or even remove the entire file.

### Directory files

- These files contain list of file names and other information related to these files.

### Special files

- These files are also known as device files.
- These files represent physical device like disks, terminals, printers, networks, tape drive etc.

These files are of two types –

- **Character special files** – data is handled character by character as in case of terminals or printers.
- **Block special files** – data is handled in blocks as in the case of disks and tapes.

## Files Organization and Access Mechanism:

File access mechanism refers to the manner in which the records of a file may be accessed. There are several ways to access files –

- Sequential access
- Direct/Random access
- Indexed sequential access

### Sequential access

A sequential access is that in which the records are accessed in some sequence, i.e., the information in the file is processed in order, one record after the other. This access method is the most primitive one. Example: Compilers usually access files in this fashion.

### Direct/Random access

- Random access file organization provides, accessing the records directly.
- Each record has its own address on the file with by the help of which it can be directly accessed for reading or writing.
- The records need not be in any sequence within the file and they need not be in adjacent locations on the storage medium.

### Indexed sequential access

- This mechanism is built up on base of sequential access.
- An index is created for each file which contains pointers to various blocks.
- Index is searched sequentially and its pointer is used to access the file directly.

## FILE ALLOCATION METHODS

**1. Continuous Allocation:** A single continuous set of blocks is allocated to a file at the time of file creation. Thus, this is a pre-allocation strategy, using variable size portions. The file allocation table needs just a single entry for each file, showing the starting block and the length of the file. This method is best from the point of view of the individual sequential file. Multiple blocks can be read in at a time to improve I/O performance for sequential processing. It is also easy to retrieve a single block. For example, if a file starts at block  $b$ , and the  $i$ th block of the file is wanted, its location on secondary storage is simply  $b+i-1$ .



File allocation table

| File name | Start block | Length |
|-----------|-------------|--------|
| File A    | 2           | 3      |
| File B    | 9           | 5      |
| File C    | 18          | 8      |
| File D    | 30          | 2      |
| File E    | 26          | 3      |

### Disadvantage

- External fragmentation will occur, making it difficult to find contiguous blocks of space of sufficient length. Compaction algorithm will be necessary to free up additional space on disk.
- Also, with pre-allocation, it is necessary to declare the size of the file at the time of creation.

**2. Linked Allocation(Non-contiguous allocation) :** Allocation is on an individual block basis. Each block contains a pointer to the next block in the chain. Again the file table needs just a single entry for each file, showing the starting block and the length of the file. Although pre-allocation is possible, it is more common simply to allocate blocks as needed. Any free block can be added to the chain. The blocks need not be continuous. Increase in file size is always possible if free disk block is available. There is no external fragmentation because only one block at a time is needed but there can be internal fragmentation but it exists only in the last disk block of file.

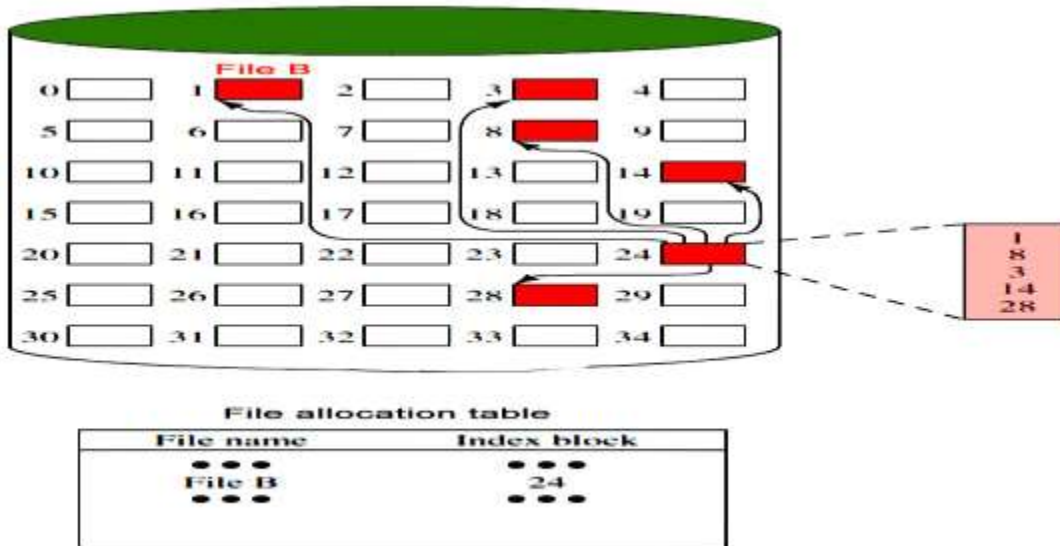
### Disadvantage:

- Internal fragmentation exists in last disk block of file.
- There is an overhead of maintaining the pointer in every disk block.
- If the pointer of any disk block is lost, the file will be truncated.
- It supports only the sequential access of files.

### 3. Indexed Allocation:

It addresses many of the problems of contiguous and chained allocation. In this case, the file allocation table contains a separate one-level index for each file: The index has one entry for each block allocated to the file. Allocation may be on the basis of fixed-size blocks or variable-sized blocks. Allocation by blocks eliminates external fragmentation, whereas allocation by variable-size blocks improves locality. This allocation technique

supports both sequential and direct access to the file and thus is the most popular form of file allocation.



Just as the space that is allocated to files must be managed, so the space that is not currently allocated to any file must be managed. To perform any of the file allocation techniques, it is necessary to know what blocks on the disk are available.

## FILE DIRECTORIES:

Collection of files is a file directory. The directory contains information about the files, including attributes, location and ownership. Much of this information, especially that is concerned with storage, is managed by the operating system. The directory is itself a file, accessible by various file management routines.

### Information contained in a device directory are:

- Name
- Type
- Address
- Current length
- Maximum length
- Date last accessed
- Date last updated
- Owner id
- Protection information

### Operation performed on directory are:

- Search for a file
- Create a file
- Delete a file
- List a directory
- Rename a file
- Traverse the file system

### Advantages of maintaining directories are:

- **Efficiency:** A file can be located more quickly.
- **Naming:** It becomes convenient for users as two users can have same name for different files or may have different name for same file.
- **Grouping:** Logical grouping of files can be done by properties e.g. all java programs, all games etc.

Let's have a look at the directory structure.

- The field File name provides the name of the concerned file in the directory.
- the Type field shows the file's type or category.
- the Location Info field gives the file's location.
- The Flag field contains information on the kind of directory entry. For example, value D indicates that the file is a directory, value L indicates that the file is a link, and value M indicates that the file is a mounted file system.
- The Protection Info field indicates whether or not the file can be viewed by other users on the system.
- The directory's Misc info file contains miscellaneous information such as the file's owner, the date it was created, and the last time it was edited.

There are many ways to organize a directory, with different levels of complexity, flexibility and efficiency.

| File name | Type/size | Location info | Protection info | Flags | Misc Info |
|-----------|-----------|---------------|-----------------|-------|-----------|
|           |           |               |                 |       |           |

## 1. Single Level Directory

The most basic way is to keep a single large list of all files on a drive. When the number of files grows or the system has more than one user, a single level directory becomes a severe constraint. No two files with the same name are allowed.

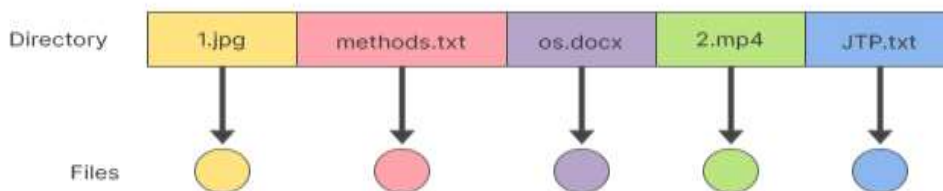
Multiple users/programs on a disc may necessitate this directory. There is no method to group files. There is only one long list. Searches must traverse the full directory.

### **Advantages**

- Because it is just a single directory, it is relatively simple to implement.
- Searching will be faster if the files are smaller in size.
- In such a directory structure, actions like file creation, searching, deletion, and updating are quite simple.

### **Disadvantages**

- In a Single-Level Directory, if the directory is vast, searching will be difficult.
- We can't group the same type of files in a single-level directory.
- The challenge of selecting a unique file name is a little more difficult.



## 2. Two-Level Directory

Another sort of directory layout is the two-level directory structure. It is feasible to create a separate directory for each user in this way. This two-level structure enables the usage of the same file name across several user directories. There is a single master directory that contains individual directories for each user. At the second level, there is a separate directory for each user, which contains a collection of users' files. The mechanism prevents a user from accessing another user's directory without their authorization.

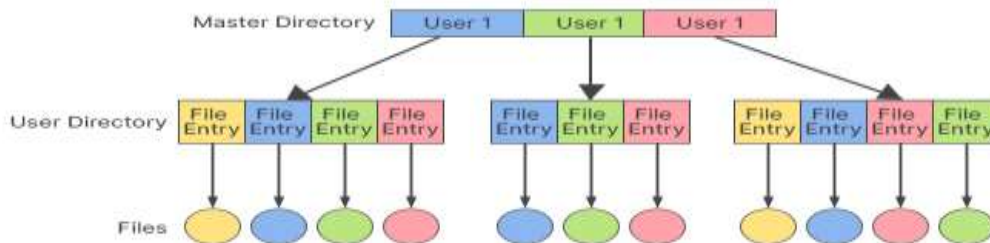
Files now have a path: /user1/directory-name. Different users can have the same file name (/user2/me and /user3/me). Because only one user's list needs to be searched, searching is more efficient. However, there is currently no way to group a user's files.

#### **Advantages**

- In a Single-Level Directory, if the directory is vast, searching will be difficult.
- We can't group files of the same type in a single-level directory.
- The challenge of selecting a unique file name is a little more difficult.

#### **Disadvantages**

- One user cannot share a file with another user in a two-level directory.
- The two-level directory also has the problem of not being scalable.
- 



### **3. Tree-structured Directory**

A file or a subsidiary can be any directory entry. In this directory structure, searching is more efficient, and the concept of a current working directory is used. Files can be arranged in logical groups. We can put files of the same type in the same directory.

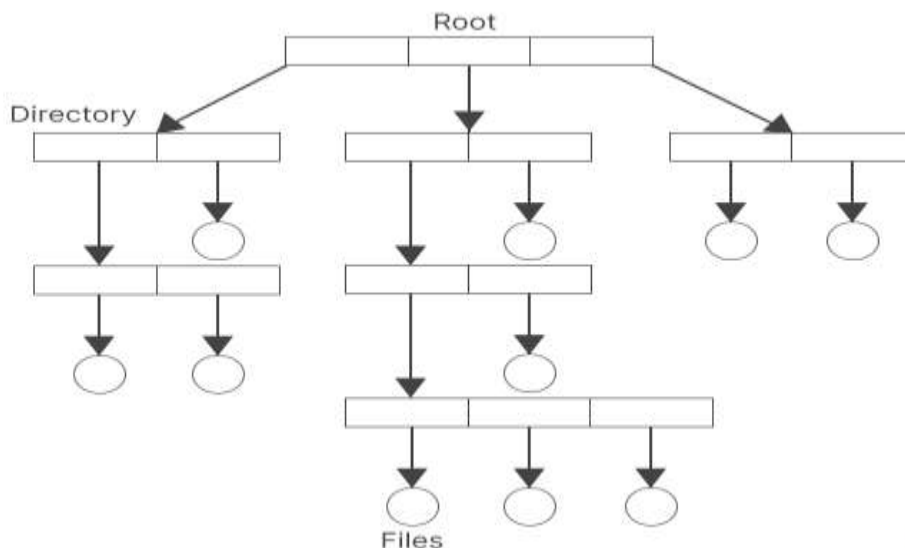
It works on files in the current directory by default, or you can give a relative or absolute path. Ability to create and delete directories, every file in the system has a unique path and the tree has a root directory.

#### **Advantages**

- The directory, which is organized in a tree form, is extremely scalable.
- Collisions are less likely in the tree-structures directory.
- The searching in the tree-structure directory is relatively simple because we may utilize both absolute and relative paths.

#### **Disadvantages**

- Files may be saved in numerous directories if they do not fit into the hierarchical model.
- We are unable to share files.
- Because a file might be found in several folders, it is inefficient.



## 4. Acyclic-Graph Directory

The tree model forbids the existence of the same file in several directories. By making the directory an acyclic graph structure, we may achieve this. Two or more directory entries can lead to the same subdirectory or file, but we'll limit it, for now, to prevent any directory entries from pointing back up the directory structure.

Links or aliases can be used to create these type of directory graph. We have numerous names for the same file, as well as multiple paths to it. There are two types of links:

- (i) symbolic link or soft link (specify a file path: logical) and
- (ii) hard link (actual link to the same file on the disc from multiple directories: physical).

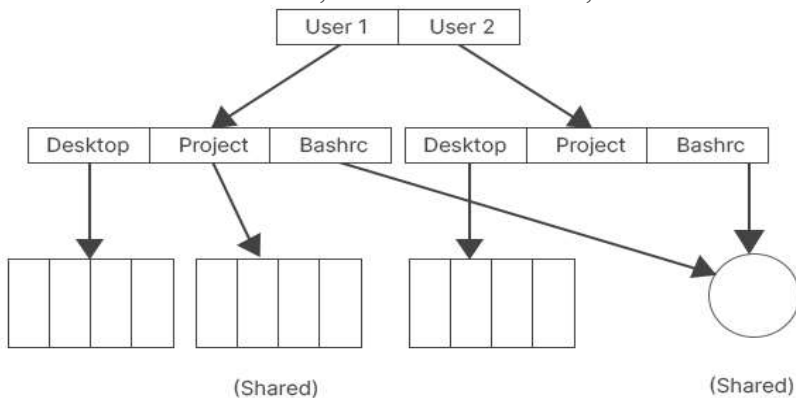
If we delete the file, there may be more references to it. The file is simply erased via symbolic links, leaving a dangling pointer. A reference count is kept through hard links, and the actual file is only erased once all references to it have been removed.

### Advantages

- We have the ability to share the files.
- Due to different-different paths, searching is simple.

### Disadvantages

- If the files are linked together, removing them may be difficult.
- If we use softlink, then if the file is removed, all that is left is a dangling pointer.
- If we use a hardlink, when we delete a file, we must also erase all of the references that are associated with it.



## 5. General Graph Directory

Cycles are allowed inside a directory structure where numerous directories can be derived from more than one parent directory in a general graph directory structure. When general graph directories are allowed, commands like search a directory and its subdirectories for something must be used with caution. If cycles are allowed, the search is infinite.

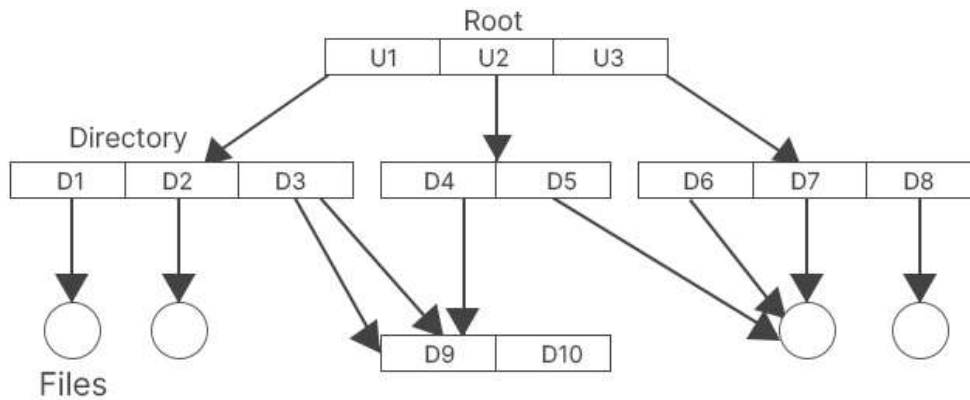
The biggest issue with this type of directory layout is figuring out how much space the files and folders have used up.

### Advantages

- The General-Graph directory structure is more adaptable than the others.
- In the general-graph directory, cycles are permitted.

### Disadvantages

- It is more expensive than other options.
- Garbage collection is required.



## Directory Implementation

An individual subdirectory will typically contain a list of files. The choice of a suitable algorithm for directory implementation is critical since it has a direct impact on system performance. Based on the data structure, we may classify the directory implementation algorithm. It can be implemented with the following approaches:

- **Linear List:** It contains a list of names, each of which has a pointer to the file's data blocks. It requires a costly search on large directories.
- **Hash Table:** It is hashed linear list, that decreases search time, but is more complex to implement. We can identify the key and key points to the relevant file that is stored in a directory using the hash function on the respective file name.

To summarise, Directory stores the entry for a set of linked files as well as information such as file names, kinds, and locations. Create, remove, list, rename, link, unlink, and other actions can be performed on directories. The directory includes files as well as information about them. It's just a folder that may be used to store and manage different files.

## File Sharing:

### Introduction

File sharing is the practice of distributing or providing access to digital files between two or more users or devices. While it is a convenient way to share information and collaborate on projects, it also comes with risks such as malware and viruses, data breaches, legal consequences, and identity theft. Protecting files during sharing is essential to ensure confidentiality, integrity, and availability. Encryption, password protection, secure file transfer protocols, and regularly updating antivirus and anti-malware software are all important measures that can be taken to safeguard files. This article will explore the different types of file sharing, risks associated with file sharing, protection measures, and best practices for secure file sharing.

File sharing refers to the process of sharing or distributing electronic files such as documents, music, videos, images, and software between two or more users or computers.

### Importance of file sharing

File sharing plays a vital role in facilitating collaboration and communication among individuals and organizations. It allows people to share files quickly and easily across different locations, reducing the need for physical meetings and enabling remote work. File sharing also helps individuals and organizations save time and money, as it eliminates the need for physical transportation of files.

## Risks and challenges of file sharing

File sharing can pose several risks and challenges, including the spread of malware and viruses, data breaches and leaks, legal consequences, and identity theft. Unauthorized access to sensitive files can also result in loss of intellectual property, financial losses, and reputational damage.

## The need for file protection

With the increase in cyber threats and the sensitive nature of the files being shared, it is essential to implement adequate file protection measures to secure the files from unauthorized access, theft, and cyberattacks. Effective file protection measures can help prevent data breaches and other cyber incidents, safeguard intellectual property, and maintain business continuity.

## Types of File Sharing

File sharing refers to the practice of distributing or providing access to digital files, such as documents, images, audio, and video files, between two or more users or devices. There are several types of file sharing methods available, and each method has its own unique advantages and disadvantages.

- **Peer-to-Peer (P2P) File Sharing** – Peer-to-peer file sharing allows users to share files with each other without the need for a centralized server. Instead, users connect to each other directly and exchange files through a network of peers. P2P file sharing is commonly used for sharing large files such as movies, music, and software.
- **Cloud-Based File Sharing** – Cloud-based file sharing involves the storage of files in a remote server, which can be accessed from any device with an internet connection. Users can upload and download files from cloud-based file sharing services such as Google Drive, Dropbox, and OneDrive. Cloud-based file sharing allows users to easily share files with others, collaborate on documents, and access files from anywhere.
- **Direct File Transfer** – Direct file transfer involves the transfer of files between two devices through a direct connection such as Bluetooth or Wi-Fi Direct. Direct file transfer is commonly used for sharing files between mobile devices or laptops.
- **Removable Media File Sharing** – Removable media file sharing involves the use of physical storage devices such as USB drives or external hard drives. Users can copy files onto the device and share them with others by physically passing the device to them.

Each type of file sharing method comes with its own set of risks and challenges. Peer-to-peer file sharing can expose users to malware and viruses, while cloud-based file sharing can lead to data breaches if security measures are not implemented properly. Direct file transfer and removable media file sharing can also lead to data breaches if devices are lost or stolen.

To protect against these risks, users should take precautions such as using encryption, password protection, secure file transfer protocols, and regularly updating antivirus and antimalware software. It is also essential to educate users on safe file sharing practices and limit access to files only to authorized individuals or groups. By taking these steps, users can ensure that their files remain secure and protected during file sharing.

## Risks of File Sharing

File sharing is a convenient and efficient way to share information and collaborate on projects. However, it comes with several risks and challenges that can compromise the confidentiality, integrity, and availability of files. In this section, we will explore some of the most significant risks of file sharing.

- **Malware and Viruses** – One of the most significant risks of file sharing is the spread of malware and viruses. Files obtained from untrusted sources, such as peer-to-peer (P2P) networks, can contain malware that can infect the user's device and compromise the security of their files. Malware and viruses can cause damage to the user's device, steal personal information, or even use their device for illegal activities without their knowledge.
- **Data Breaches and Leaks** – Another significant risk of file sharing is the possibility of data breaches and leaks. Cloud-based file sharing services and P2P networks are particularly vulnerable to data breaches if security measures are not implemented properly. Data breaches can result in the loss of sensitive



information, such as personal data or intellectual property, which can have severe consequences for both individuals and organizations.

- **Legal Consequences** – File sharing copyrighted material without permission can lead to legal consequences. Sharing copyrighted music, movies, or software can result in copyright infringement lawsuits and hefty fines.
- **Identity Theft** – File sharing can also expose users to identity theft. Personal information, such as login credentials or social security numbers, can be inadvertently shared through file sharing if security measures are not implemented properly. Cybercriminals can use this information to commit identity theft, which can have severe consequences for the victim.

To protect against these risks, users should take precautions such as using trusted sources for file sharing, limiting access to files, educating users on safe file sharing practices, and regularly updating antivirus and anti-malware software. By taking these steps, users can reduce the risk of malware and viruses, data breaches and leaks, legal consequences, and identity theft during file sharing.

## File Sharing Protection Measures

- **Encryption** – Encryption is the process of converting data into a coded language that can only be accessed by authorized users with a decryption key. This can help protect files from unauthorized access and ensure that data remains confidential even if it is intercepted during file sharing.
- **Password protection** – Password protection involves securing files with a password that must be entered before the file can be accessed. This can help prevent unauthorized access to files and ensure that only authorized users can view or modify the files.
- **Secure file transfer protocols** – Secure file transfer protocols, such as SFTP (Secure File Transfer Protocol) and HTTPS (Hypertext Transfer Protocol Secure), provide a secure way to transfer files over the internet. These protocols use encryption and other security measures to protect files from interception and unauthorized access during transfer.
- **Firewall protection** – Firewall protection involves using a firewall to monitor and control network traffic to prevent unauthorized access to the user's device or network. Firewalls can also be configured to block specific file sharing protocols or limit access to certain users or devices, providing an additional layer of protection for shared files.

## File system Implementation

### Introduction

The file system is an integral part of any operating system that provides a structured way of storing and accessing files. The implementation of a file system involves designing and developing software components that manage the organization, allocation, and access to files on the disk. This includes managing the physical storage devices, keeping track of file attributes and metadata, enforcing security and access control policies, and providing utilities for managing files and directories. The file system implementation plays a crucial role in ensuring the reliability, performance, and security of an operating system's file storage capabilities, and it requires sophisticated software design and engineering techniques to achieve these goals.

File system implementation is the process of designing, developing, and implementing the software components that manage the organization, allocation, and access to files on a storage device in an operating system.

### Importance of file system implementation

The file system implementation plays a critical role in ensuring the reliability, performance, and security of an operating system's file storage capabilities. Without an effective file system implementation, an operating system cannot efficiently manage the storage of data on a storage device, resulting in data loss, corruption, and inefficiency.

### File System Structure

- Disk layout and partitioning
- File system organization

- File allocation methods
- Directory structure

### **Disk layout and partitioning**

The disk layout and partitioning refers to how a physical disk is divided into logical partitions, which can be accessed by the operating system as separate entities. The disk is divided into one or more partitions which can be formatted with a file system. Disk partitioning involves creating partitions on the disk, while disk formatting involves creating a file system on the partition. The partitioning process is typically done when the disk is first installed, and the formatting process is typically done when a partition is created.

### **File system organization**

The file system organization refers to how files and directories are stored on the disk. A file system is responsible for managing files and directories, and providing a way for users and applications to access and modify them. Different file systems may organize files and directories in different ways, and may use different methods for storing and accessing them. For example, the FAT file system used by Windows organizes files in a simple directory hierarchy, while the HFS+ file system used by macOS organizes files in a more complex tree structure.

### **File allocation methods**

The file allocation method refers to how file data is stored on the disk. There are several different file allocation methods, including contiguous allocation, linked allocation, and indexed allocation. Contiguous allocation stores files in contiguous blocks on the disk, while linked allocation uses pointers to link blocks of data together. Indexed allocation uses an index to keep track of where each file block is stored on the disk.

### **Directory structure**

The directory structure refers to how directories are organized and managed on the disk. Directories are used to organize files and other directories into a hierarchy, which can be navigated by users and applications. Different file systems may use different directory structures, including single-level directories, two-level directories, and tree-structured directories. Directories can also have various attributes such as permissions and ownership, which can control who can access and modify files within them.

## **File System Operations**

- File creation and deletion
- File open and close
- File read and write
- File seek and position
- File attributes and permissions

### **File creation and deletion**

File creation involves allocating space on the disk for a new file and setting up its attributes and permissions. File deletion involves removing the file from the disk and releasing the space it occupies. In some file systems, deleted files may be recoverable if they have not been overwritten.

### **File open and close**

File open involves establishing a connection between the file and a process or application that wishes to access it. File close involves terminating that connection and freeing up any resources used by the process or application.

### **File read and write**

File read involves retrieving data from a file and transferring it to a process or application. File write involves sending data from a process or application to a file. These operations can be performed at various levels of granularity, such as bytes, blocks, or sectors.

### **File seek and position**

File seek involves moving the current position of the file pointer to a specific byte or block within the file. File position refers to the current location of the file pointer within the file. These operations are useful for random access and manipulation of specific portions of a file.

## **File attributes and permissions**

File attributes refer to metadata associated with a file, such as its name, size, and creation/modification dates. File permissions refer to the access control settings that determine who can read, write, execute, or modify a file. These settings can be set for individual users or groups, and can be used to restrict access to sensitive data or programs.

Each of these file system operations is essential for managing files and directories on a computer or network. The implementation of these operations may vary depending on the type of file system and the operating system being used.

## **Implementation Issues**

- Disk space management
- Consistency checking and error recovery
- File locking and concurrency control
- Performance optimization

### **Disk space management**

File systems need to manage disk space efficiently to avoid wasting space and to ensure that files can be stored in contiguous blocks whenever possible. Techniques for disk space management include free space management, fragmentation prevention, and garbage collection.

### **Consistency checking and error recovery**

File systems need to ensure that files and directories remain consistent and error-free. Techniques for consistency checking and error recovery include journaling, checksumming, and redundancy. If errors occur, file systems may need to perform recovery operations to restore lost or damaged data.

### **File locking and concurrency control**

File systems need to manage access to files by multiple processes or users to avoid conflicts and ensure data integrity. Techniques for file locking and concurrency control include file locking, semaphore, and transaction management.

### **Performance optimization**

File systems need to optimize performance by reducing file access times, increasing throughput, and minimizing system overhead. Techniques for performance optimization include caching, buffering, prefetching, and parallel processing.

These implementation issues are critical for ensuring that file systems operate efficiently, reliably, and securely. File system designers must carefully balance these factors to create a system that meets the needs of its users and the applications that use it.

## **File System Protection and Security**

Protection and security requires that computer resources such as CPU, softwares, memory etc. are protected. This extends to the operating system as well as the data in the system. This can be done by ensuring integrity, confidentiality and availability in the operating system. The system must be protect against unauthorized access, viruses, worms etc.

### **Protection in File System**

In computer systems, alot of user's information is stored, the objective of the operating system is to keep safe the data of the user from the improper access to the system. Protection can be provided in number of ways. For a single laptop system, we might provide protection by locking the computer in a desk drawer or file cabinet. For multi-user systems, different mechanisms are used for the protection.

## Types of Access :

The files which have direct access of the any user have the need of protection. The files which are not accessible to other users doesn't require any kind of protection. The mechanism of the protection provide the facility of the controlled access by just limiting the types of access to the file. Access can be given or not given to any user depends on several factors, one of which is the type of access required. Several different types of operations can be controlled:

- **Read** – Reading from a file.
- **Write** – Writing or rewriting the file.
- **Execute** – Loading the file and after loading the execution process starts.
- **Append** – Writing the new information to the already existing file, editing must be end at the end of the existing file.
- **Delete** – Deleting the file which is of no use and using its space for the another data.
- **List** – List the name and attributes of the file.

Operations like renaming, editing the existing file, copying; these can also be controlled. There are many protection mechanism. each of them mechanism have different advantages and disadvantages and must be appropriate for the intended application.

## Access Control :

There are different methods used by different users to access any file. The general way of protection is to associate *identity-dependent access* with all the files and directories an list called access-control list (ACL) which specify the names of the users and the types of access associate with each of the user. The main problem with the access list is their length. If we want to allow everyone to read a file, we must list all the users with the read access. This technique has two undesirable consequences:

Constructing such a list may be tedious and unrewarding task, especially if we do not know in advance the list of the users in the system.

Previously, the entry of the any directory is of the fixed size but now it changes to the variable size which results in the complicates space management. These problems can be resolved by use of a condensed version of the access list. To condense the length of the access-control list, many systems recognize three classification of users in connection with each file:

- **Owner** – Owner is the user who has created the file.
- **Group** – A group is a set of members who has similar needs and they are sharing the same file.
- **Universe** – In the system, all other users are under the category called universe.

The most common recent approach is to combine access-control lists with the normal general owner, group, and universe access control scheme. For example: Solaris uses the three categories of access by default but allows access-control lists to be added to specific files and directories when more fine-grained access control is desired.

## Other Protection Approaches:

The access to any system is also controlled by the password. If the use of password are is random and it is changed often, this may be result in limit the effective access to a file.

The use of passwords has a few disadvantages:

- The number of passwords are very large so it is difficult to remember the large passwords.
- If one password is used for all the files, then once it is discovered, all files are accessible; protection is on all-or-none basis.

## Threats to Protection and Security

A threat is a program that is malicious in nature and leads to harmful effects for the system. Some of the common threats that occur in a system are –

**Virus :** Viruses are generally small snippets of code embedded in a system. They are very dangerous and can corrupt files, destroy data, crash systems etc. They can also spread further by replicating themselves as required.

**Trojan Horse:** A trojan horse can secretly access the login details of a system. Then a malicious user can use these to enter the system as a harmless being and wreak havoc.

**Trap Door:** A trap door is a security breach that may be present in a system without the knowledge of the users. It can be exploited to harm the data or files in a system by malicious people.

**Worm:** A worm can destroy a system by using its resources to extreme levels. It can generate multiple copies which claim all the resources and don't allow any other processes to access them. A worm can shut down a whole network in this way.

**Denial of Service:** These type of attacks do not allow the legitimate users to access a system. It overwhelms the system with requests so it is overwhelmed and cannot work properly for other user.

## Protection and Security Methods

The different methods that may provide protect and security for different computer systems are –

### Authentication

This deals with identifying each user in the system and making sure they are who they claim to be. The operating system makes sure that all the users are authenticated before they access the system. The different ways to make sure that the users are authentic are:

- **Username/ Password:** Each user has a distinct username and password combination and they need to enter it correctly before they can access the system.
- **User Key/ User Card:** The users need to punch a card into the card slot or use they individual key on a keypad to access the system.
- **User Attribute Identification:** Different user attribute identifications that can be used are fingerprint, eye retina etc. These are unique for each user and are compared with the existing samples in the database. The user can only access the system if there is a match.

### One Time Password

These passwords provide a lot of security for authentication purposes. A one time password can be generated exclusively for a login every time a user wants to enter the system. It cannot be used more than once. The various ways a one time password can be implemented are –

- **Random Numbers:** The system can ask for numbers that correspond to alphabets that are pre arranged. This combination can be changed each time a login is required.
- **Secret Key:** A hardware device can create a secret key related to the user id for login. This key can change each time.

## Seek time:

- The seek time of a hard disk measures the amount of time required for the read/write heads to move between tracks over the surfaces of the platters.
- Seek time is normally expressed in milliseconds (commonly abbreviated "msec" or "ms"), with average seek times for most modern drives today in a rather tight range of 8 to 10 ms.

### Different types of seeks:

- **Average:** As discussed, this is meant to represent an average seek time from one random track (cylinder) to any other. This is the most common seek time metric, and is usually 8 to 10 ms, though older drives had much higher numbers, and top-of-the-line SCSI drives are now down to as low as 4 ms!
- **Track-to-Track:** This is the amount of time that is required to seek between adjacent tracks. This is similar in concept (but not exactly the same as) the track switch time and is usually around 1ms. (Incidentally, getting this figure without at least two significant digits is pretty meaningless; don't accept "1 ms" for an answer, get the number after the decimal point! Otherwise every drive will probably round off to "1 ms".)
- **Full Stroke:** This number is the amount of time to seek the entire width of the disk, from the innermost track to the outermost. This is of course the largest number, typically being in the 15 to 20 ms range. In

some ways, combining this number with the average seek time represents the way the drive will behave when it is close to being full.

## Rotational Latency:

- Rotational latency is the delay waiting for the rotation of the disk to bring the required disk sector under the read-write head.
- It depends on the rotational speed of a disk (or spindle motor), measured in revolutions per minute(RPM).For most magnetic media-based drives, the average rotational latency is typically based on the empirical relation that the average latency in milliseconds for such a drive is one-half the rotational period.

### Two types of disk rotation methods:

1) constant linear velocity (CLV), used mainly in optical storage, varies the rotational speed of the optical disc depending upon the position of the head, and

2) Constant angular velocity (CAV), used in HDDs, standard FDDs, a few optical disc systems, and vinyl audio records, spins the media at one constant speed regardless of where the head is positioned.

- To access data, the actuator arm moves the R/W head over the platter to a particular track while the platter spins to position the requested sector under the R/W head. The time taken by the platter to rotate and position the data under the R/W head is called rotational latency. This latency depends on the rotation speed of the spindle and is measured in milliseconds. The average rotational latency is one-half of the time taken for a full rotation. Similar to the seek time, rotational latency has more impact on the reading/writing of random sectors on the disk than on the same operations on adjacent sectors. Average rotational latency is around 5.5 ms for a 5,400-rpm drive, and around 2.0 ms for a 15,000-rpm drive.

### Utilization:

- Utilization factor or use factor is the ratio of the time that a piece of equipment is in use to the total time that it could be in use. It is often averaged over time in the definition such that the ratio becomes the amount of energy used divided by the maximum possible to be used.
- Utilization can be defined as the ratio of the service time to the average inter-arrival time, and is expressed as:  $U = RS / Ra$  Where “RS” is the service time, or the average time spent by a request on the controller.  $1/RS$  is the service rate.